TATA TECHNOLOGIES LTD.

TECH PULSE FY-26 | AI/ML Capstone Project

PROJECT SYNOPSIS

Student Details

Institute/University: Pimpri Chinchwad College of Engineering (PCCOE)

Programme / Batch: B.Tech. CS-AIML, 2027 Batch

Student Name: Om Deshmukh

PRN: 123B1C004

1. Case Study ID and Project Title / Topic

Case Study ID (CS1 to CS5): CS1

Project Title / Topic: AutoDoc AI — Automotive Architecture Assistant for HLD Document Question Answering

2. Objectives

Objectives

- • Allow users to upload automotive High-Level Design (HLD) documents in supported PDF or TXT formats.
- • Extract and preprocess document text, then divide it into manageable chunks for retrieval.
- • Retrieve passages relevant to a user question using TF-IDF and cosine similarity.
- • Present responses with supporting source passages to help users verify information against the document.

3. Scope

In Scope – To Be Implemented

- • Streamlit-based interface for document upload and natural-language questions.
- • PDF/TXT text extraction, text chunking, and lexical retrieval using TF-IDF/cosine similarity.
- • Display relevant retrieved passages and provide a fallback response when configured generation is unavailable.
- • Test using the included synthetic automotive HLD and a prepared question set; record actual evaluation results.

Out of Scope – Will Not Be Implemented

- • No use of confidential OEM, customer, vehicle, or personal data; use synthetic or explicitly approved documents only.
- • No claim of official AUTOSAR compliance, production readiness, or safety validation.
- • No autonomous engineering decisions, vehicle control, or replacement of expert human review.

4. Proposed Approach

Pre-trained model (name, version, local/private hosting)

Pre-trained model: Optional local Ollama model only if installed and configured; record exact model name/version used during testing. Core retrieval does not require a generative model.

If no local model is configured, use the application's available retrieval/extractive fallback; do not report an LLM as used unless verified.

Embedding model

Embedding model: Not used in the current baseline. TF-IDF provides sparse lexical vectors for document chunks and questions.

Future extension: evaluate a sentence-embedding model only if approved and implemented.

Vector store (FAISS / ChromaDB)

Vector store: No FAISS/ChromaDB vector database in the current baseline; TF-IDF vectors are compared in the retrieval module.

This keeps the prototype lightweight and avoids implying that a semantic embedding store has been implemented.

RAG pipeline and/or AI-agent design – brief description

Upload document → extract text → split into chunks → calculate TF-IDF representations → rank chunks by cosine similarity to the question → return relevant evidence and a response.

Optional local LLM generation may use retrieved context when configured; answers must be checked against the displayed source evidence.

Service layer and UI (e.g., FastAPI / Flask / Streamlit)

Service layer/UI: Streamlit application (Code/app.py). The project uses a local interactive web interface rather than a separate FastAPI/Flask service.

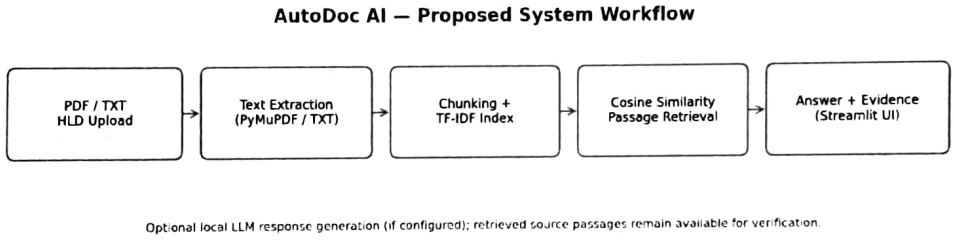
Core retrieval and evaluation logic are separated into Python modules (including Code/retrieval.py and Code/evaluate.py).

Tools / deterministic rules, if applicable

Deterministic rules: file-type handling, text extraction, chunking, TF-IDF vectorization, cosine-similarity ranking, and evidence display.

When document evidence is insufficient, the user should treat the answer as unknown and verify the source rather than infer technical facts.

Proposed System / Architecture



5. Input Data / Knowledge Base

Use approved or synthetic data only.

Data / Knowledge Sources

Synthetic automotive HLD text included in Input_Data; prepared question set for functional and retrieval evaluation.

No official AUTOSAR, OEM, customer, or proprietary documents are included. Any additional data must be synthetic or explicitly approved.

Approximate Size and Format

Primary input: TXT synthetic HLD; optional user-supplied PDF/TXT documents with selectable text. Evaluation questions are stored in a CSV file.

Dataset size is limited and intended for a student prototype; report the actual file size and number of chunks/questions from the submitted files.

Preprocessing and EDA Planned

Normalize extracted text, remove irrelevant whitespace where appropriate, split text into chunks, and build TF-IDF features for retrieval.

Inspect sample questions and retrieved passages manually; check extraction quality, relevance, and whether the answer is supported by evidence.

6. Expected Outcomes and Evaluation Plan

Expected Outputs – aligned to the case study

A working document-question-answering prototype, retrieved evidence passages, and a repeatable evaluation script/question set.

Technical report, setup/user documentation, demonstration video, and measured evaluation results to be included in the final submission.

Evaluation Method – test question set / ground truth / metrics / validation approach

Use the prepared test-question set; compare retrieved passages and answers against the source HLD/manual ground truth. Report retrieval success and answer correctness using observed results.

Run Code/evaluate.py and manually inspect representative successes and failures. Do not fill metric values until the script has been run and outputs verified.

Responsible AI Measures – grounding and citations / privacy / security / human oversight

Use synthetic/approved data, display source evidence, avoid unsupported claims, and require human verification of technical responses.

Do not upload confidential material. Treat outputs as educational assistance only; no production automotive or safety decisions should rely on the prototype.

MLOps / Deployment Elements Planned – versioning / Docker / tracking

Version control: maintain named project files and document changes. Deployment: local Streamlit run; package dependencies in requirements.txt.

Docker, experiment tracking, and automated CI/CD are not claimed as implemented unless separately added and tested.

7. Planned AI-Tool Usage

Mention the AI tools / models used for coding assistance and specify the purpose and project artifact affected. This information will be carried into the final AI-tool usage declaration.

AI Tool / Model

ChatGPT (OpenAI) — coding and documentation assistance; disclose actual use and review all generated material.

Purpose

Support brainstorming, code troubleshooting, explanation of implementation choices, and drafting/editing project documentation.

Artifact Affected

Python application files, setup/user guides, synopsis, technical/evaluation reports, and demo script as applicable. Student must verify, understand, and own submitted work.

Student Declaration

☑ I will not change the approved case-study scope, major methodology, or implementation approach without prior faculty approval.

☑ I confirm that the work submitted under this synopsis represents my own understanding and contribution, and I will be responsible for explaining every significant part of the selected case study and its implementation.

Name and Signature: Om Deshmukh

Date: 7/10/2026